

FIRS Sampling Exam

Sampling Exercise

Planning for Zürichberg Post-Storm Remeasurement

The Zürichberg forest was inventoried the year before a major winter storm. The historical inventory contains measurements of stem number and basal area on field plots.

The forest owner now wants to know:

1. the total number of stems affected by the storm, and
2. the mean basal area affected by the storm.

Field remeasurement is expensive, so only a sample of the old plots can be remeasured. Your task is to propose **three different probability sampling designs**. For each design:

- explain how the sample will be selected and the reasoning for implementing it;
 - SRS without replacement
 - Two-stage sampling
 - cluster sampling
 - Stratification
 - etc...
- draw a realization of the sample to show to the owner;
- specify the estimator of total affected stems;
- specify the estimator of mean affected basal area;
- explain how the estimator will look and how uncertainty would be estimated;
- discuss advantages and disadvantages from the owner's perspective.

The owner doesn't know anything about sampling and will choose among your proposals so you will want to express yourself visually.

The dataset is loaded as follows:

```
library(forestinventory)
data(zberg)

zberg_forest <- zberg[!is.na(zberg$basal), ]
```

The package documentation describes **stem** as field-derived stem number and **basal** as field-derived basal area. It also provides **cluster**, **stade**, **melange**, and **couver** as potentially useful auxiliary information.

Estimation Exercise

This exercise is a continuation of the Monte Carlo integration example discussed in class. In that example, we considered estimating an integral using randomly generated points. Here, we consider a three-dimensional “local density surface” that we want to integrate over a two-dimensional domain.

The function is defined as follows:

```
Y <- function(x1, x2, true_beta = c(30, 13, -6, -4, 3, 2)){
  Z_x <- c(1, x1, x2, x1^2, x1*x2, x2^2)
  noise <- 6*cos(pi*x1)*sin(2*pi*x2)
  out <- t(Z_x) %*% true_beta + noise
  return(as.numeric(out))
}

# Example
Y(.5, .5)
```

The domain of integration is

$$0 \leq x_1 \leq 3, \quad 0 \leq x_2 \leq 4.$$

The surface can be visualized using:

```
library(plotly)

# Grid of x1 and x2 values
x1 <- seq(0, 3, length.out = 100)
x2 <- seq(0, 4, length.out = 100)

# Evaluate Y on the grid
z <- outer(x1, x2, Vectorize(Y))

# 3D surface
plot_ly(
  x = x1,
  y = x2,
  z = z
) |>
  add_surface() |>
  layout(
    scene = list(
      xaxis = list(title = "x1"),
      yaxis = list(title = "x2"),
      zaxis = list(title = "Y(x1, x2)")
    )
  )
```

We can obtain a numerical value for the integral using the **cubature** package:

```
library(cubature)
result <- adaptIntegrate(
  f = function(x) {Y(x[1], x[2])},
  lowerLimit = c(0, 0),
  upperLimit = c(3, 4)
)
result$integral
# 542
```

This value will be treated as the true value of the integral for purposes of this exercise. We define the auxiliary variables

$$\mathbf{Z}(x) = (x_1, x_2, x_1^2, x_1 x_2)^\top.$$

Your task is to consider **at least 3** the following estimation strategies:

1. **One-phase estimator with uniform i.i.d. sampling.**
2. **One-phase estimator with systematic sampling.**
3. **Stratified one-phase estimator with proportional uniform i.i.d. sampling.**
4. **Post-stratification using the same strata as the stratification estimator.** (Hint: this is Y_{reg} of a categorical variable). Use the plug-in variance estimator

For the stratification we partition the rectangular domain $\mathcal{D} = [0, 3] \times [0, 4]$ into four strata by bisecting each coordinate axis. The x_1 -axis is split at its midpoint $x_1 = 1.5$ and the x_2 -axis at its midpoint $x_2 = 2$. The four strata are then defined as

$$\mathcal{S}_1 = [0, 1.5] \times [0, 2), \tag{1}$$

$$\mathcal{S}_2 = [1.5, 3] \times [0, 2), \tag{2}$$

$$\mathcal{S}_3 = [0, 1.5] \times [2, 4], \tag{3}$$

$$\mathcal{S}_4 = [1.5, 3] \times [2, 4], \tag{4}$$

Thus, each strata proportion is $p_h = \frac{1}{4}$ for $h = 1, 2, 3, 4$.

```
get_stratum <- function(x1, x2) {
  ifelse(x2 < 2,
    ifelse(x1 < 1.5, "S1", "S2"), # lower half of x2
    ifelse(x1 < 1.5, "S3", "S4") # upper half of x2
  )
}
```

For each strategy, use a sample size of $n = 50$ and calculate the point estimate, the variance estimate, the confidence interval, and whether or not the true value is in the estimated confidence interval for that sample realization.

This is a Monte Carlo simulation, so we will repeat the procedure 500 times to calculate:

- the population variance, i.e., the variance of the replicate point estimates;
- the expectation of our variance estimator, i.e., the mean of the variance estimates;
- the empirical coverage rate, i.e., the mean of the binary variable indicating whether or not our confidence interval contained the true value.

Make a table with each estimation strategy as a row and columns for the true value, the expectation of the estimator, the population variance, the expectation of the variance estimator, and the empirical coverage rate.

Comment on the efficiency of the different approaches. Think about what you would expect to happen if we did this again for larger sample sizes of n .